



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER II

2024/2025

SECR 2043 – OPERATING SYSTEM

SECTION 03

GROUP: Ot

**PROJECT REPORT - REAL WORLD APPLICATION OF SYNCHRONIZATION
USING SEMAPHORE**

LECTURER: DR. DANLAMI GABI

NAME	MATRIC NUMBER
NURUL IKA SYAFINY BINTI AZHAR	A23CS0164
LUBNA AL HAANI BINTI RADZUAN	A23CS0107
NURAI SYAH BINTI MOHD ZIKRE	A23CS0160
NUR FIRZANA BINTI BADRUS HISHAM	A23CS0156

TABLE OF CONTENT

ABSTRACT	2
1.0 INTRODUCTION	3
2.0 PROBLEM STATEMENT	3
3.0 RELATED WORKS	3
4.0 METHODOLOGY	4
4.1 Project Management Flowchart	4
4.2 Hardware and Software Requirements	5
4.3 Task Distribution	5
5.0 DESIGN AND IMPLEMENTATION	6
5.1 Animation Flowchart	6
5.2 Animation Implementation	7
6.0 RESULTS AND DISCUSSION	10
6.1 Interface Execution	10
6.2 Output Analysis	11
6.3 Discussion	11
7.0 FUTURE WORK AND CONCLUSION	12
7.1 Future Work	12
7.2 Conclusion	12
8.0 ACKNOWLEDGEMENT	13
9.0 REFERENCES	14
10.0 YOUTUBE VIDEO LINK	14

ABSTRACT

This project demonstrates the concept of synchronization using semaphores through a real-world inspired simulation: managing limited toilet stalls in a public washroom, it simulates multiple users concurrently accessing a limited number of toilet stalls. Through the development of animation this report illustrates how semaphores effectively manage shared resources in a multi-process environment. Each user is modeled as a thread, and a counting semaphore regulates access to a limited number of stalls. The objective for this simulation ensures that no more users than available stalls can enter simultaneously, where only one person can occupy a stall at a time, and others must wait if all stalls are occupied. The animation, designed using Canva and edited with CapCut, illustrates key semaphore operations such as **acquire()** and **release()** by personifying the semaphore as a security guard who controls access to the stalls. It is to prevent race conditions and ensure proper resource allocation, highlighting the mutual exclusion and resource management features of semaphores. This project will detail the problem, methodology, design, implementation, and analysis of the simulation, showcasing the practical application of semaphore for synchronization which will effectively visualize core Operating System concepts in a relatable and engaging manner.

1.0 INTRODUCTION

Operating systems are tasked with managing various and multiple resources and processes efficiently. Hence, it is important to ensure multiple processes access shared resources without interfering with each other and to prevent the possibility of inconsistent data due to the concurrent access. In order to achieve this goal, various synchronization techniques such as semaphores are used as it helps manage how different processes (or programs) share resources, like memory or data, without causing conflicts, (Jain, 2025). Semaphores are widely used synchronization tools that help manage concurrent access to resources as the process can attempt to access one instance, and if it is not available, they can try other instances, (Jain, 2025). This project will simulate a public toilet with limited stalls, to provide a better understanding of semaphores concept through the development of animation.

2.0 PROBLEM STATEMENT

In a shared bathroom with multiple stalls but no locks or keys, multiple people may enter the same stall at the same time, leading to conflicts and uncomfortable situations. This scenario illustrates the problem faced by operating systems when multiple processes access a shared resource simultaneously without any control mechanism, resulting in race conditions, data inconsistency, or unexpected behavior. The solution to this problem is simple in the real world which is installing locks and providing keys so that only one person can use a stall at a time. This project aims to demonstrate this solution to explain how semaphores work to solve the synchronization problem by using the bathroom stalls analogy.

3.0 RELATED WORKS

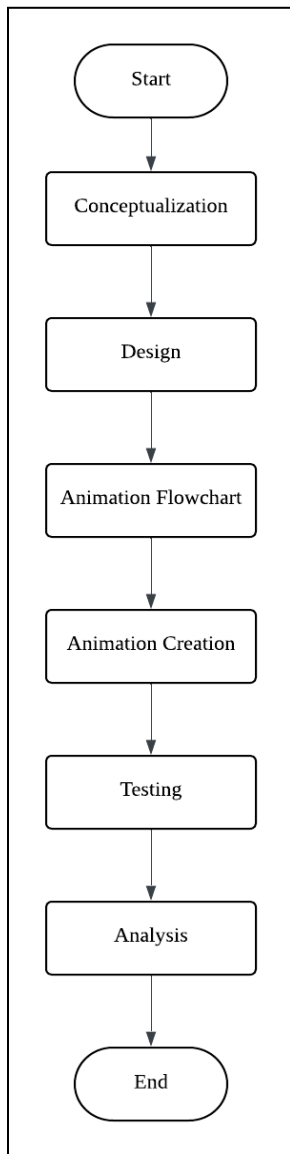
Semaphores have been widely explained in books and online, such as *The Little Book of Semaphores* by Allen B. Downey, which uses the producer-consumer problem, and websites like Arm Limited and Oracle, which use printer and railroad analogies. However, a bathroom analogy is not commonly used. Therefore, this project will explain and present an animation that illustrates our solution, using a simple bathroom example to show how semaphores control access to solve synchronization problems.

4.0 METHODOLOGY

The methodology in this section is helping to outline the phases in creating the semaphore concept to ensure the project is systematically developed.

4.1 Project Management Flowchart

The Project Management Flowchart illustrates the phases and the steps involved in this project, focusing on the semaphore concept using a real-world toilet stall scenario. It also includes the key stages like conceptualization, design, storyboard flowchart, animation creation and testing and analysis.



Steps in Flowchart:

- 1. Conceptualization:** Brainstorming and finalized the real-world situation; a Toilet Stall, to represent the mutual exclusion by using semaphore synchronization.
- 2. Design:** Planned the sequence and dialogue through the storyboard. Each of the scenes illustrates how semaphores manage access to a critical section.
- 3. Animation Flowchart:** Created the visual flow of the animation to understand the semaphore concept.
- 4. Animation Creation:** Using Canva to design the animation, including the character movements, text overlays and visual cues to represent the semaphore concept.
- 5. Testing:** Validate the animation to ensure all the story matches as expected.
- 6. Analysis:** Evaluate the results of the analogy to measure the performance improvements.

4.2 Hardware and Software Requirements

- **Software:** Canva, Capcut, LucidChart
- **Hardware:** Laptop/tablet with strong internet connection, webcam for demo

4.3 Task Distribution

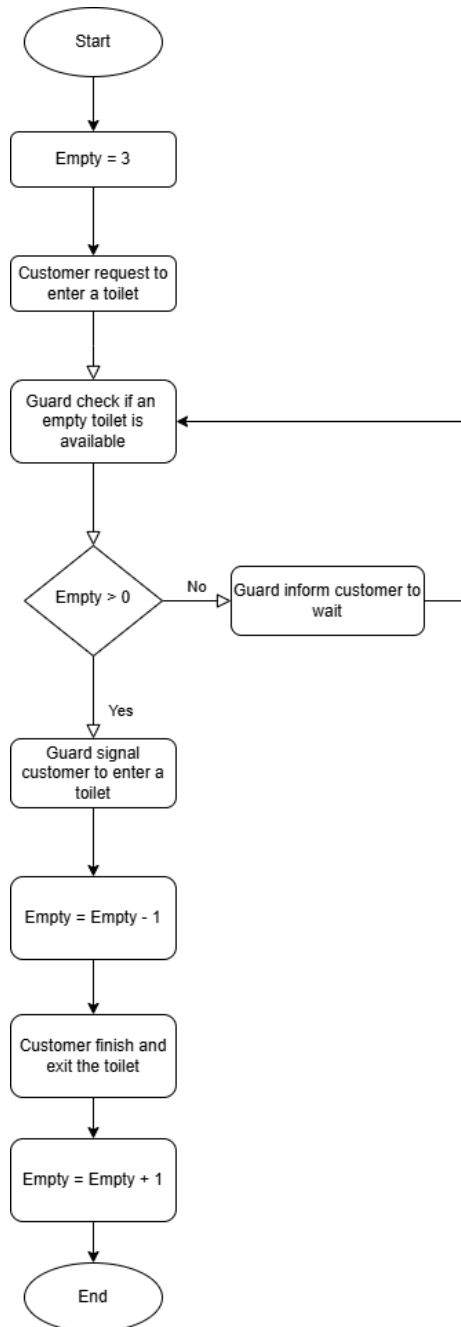
- **Concept:** Lubna Al Haani
- **Animation Design:** Nur Firzana
- **Testing and Analysis:** Nurul Ika Syafiny & Nuraisyah
- **Report Writing:** All Members

5.0 DESIGN AND IMPLEMENTATION

In this part, we are going to design the animation that will simulate the use of semaphore in real-world application starting from flowchart and then the implementation.

5.1 Animation Flowchart

First, we are going to design how the animation will simulate semaphore application in the real-world through the flowchart below.



Step in the Flowchart:

1. We initialize a variable called **Empty** to **3**, which means there are **3 toilets available** at the beginning. This is like initializing a **semaphore value** to 3, one unit for each toilet.

2. A customer arrives and wants to use a toilet. This simulates a **process requesting access** to a critical resource.

3. The **guard** plays the role of the **semaphore controller**. He checks whether any toilet is currently free ($\text{Empty} > 0$).

4. If **no toilets are available** ($\text{Empty} == 0$), the guard tells the customer to **wait**. This loops back to checking again until a toilet is free. This simulates a process being blocked or waiting on a semaphore. If **yes** ($\text{Empty} > 0$), the guard allows the customer to proceed.

5. Customers that can proceed will be signal by the guard that they may enter. This is like **P (wait)** operation in semaphore logic: permission is granted.

6. One toilet is now occupied, so we **decrease** the **Empty** count by 1. This ensures we keep track of how many toilets are left.

7. Once done, the customer **leaves the toilet**, making it available again. This is the end of the **critical section**.

8. We **increase** **Empty** by 1 to reflect that one toilet is now free. This is like the **V (signal)** operation in semaphore logic.

5.2 Animation Implementation

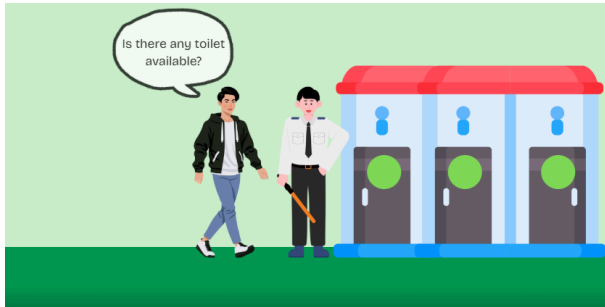
In this section, we are going to apply the flowchart into animation and give a brief explanation for each animation cut.

Initialize available toilet = 3



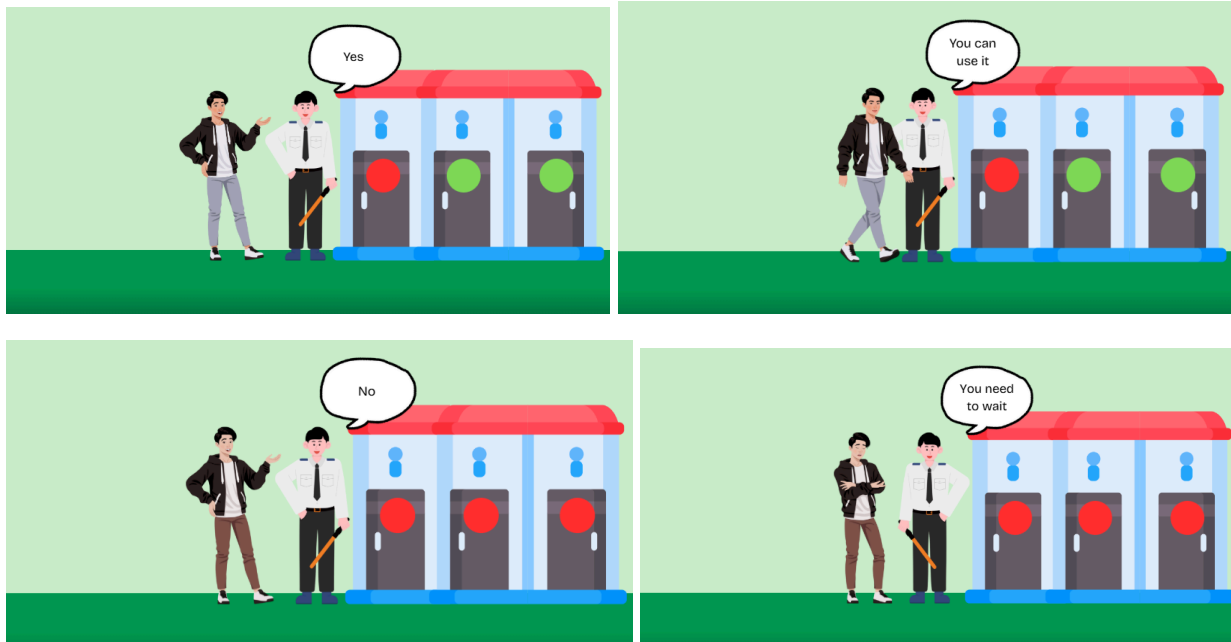
We initialize a variable called Empty to 3, which means there are 3 toilets available at the beginning that can be used by customers. This is like initializing a semaphore value to 3, one unit for each toilet.

Customer request to enter a toilet



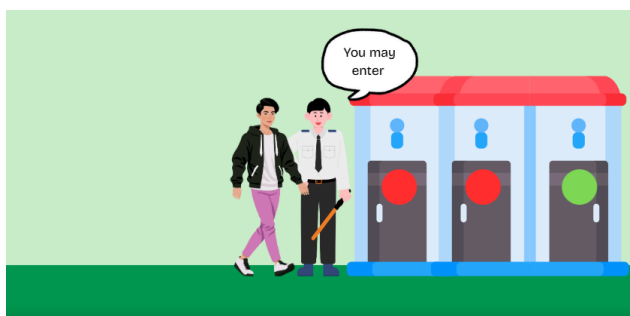
A customer arrives and wants to use a toilet. This simulates a process requesting access to a critical resource. They have to ask the guard first before being allowed to enter the toilet.

Guard check if empty toilet is available



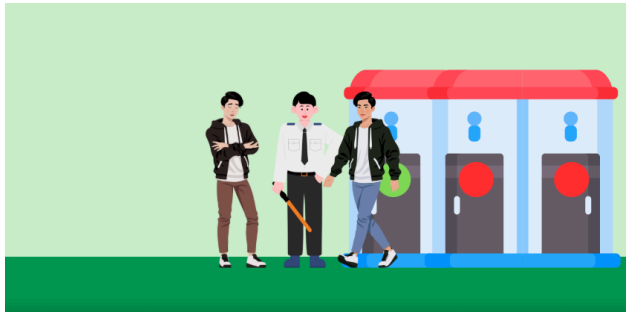
Before a customer can enter the toilet, the guard will check first if there is any available toilet. The guard plays the role of the semaphore controller. He checks whether any toilet is currently free ($\text{Empty} > 0$). If no toilets are available ($\text{Empty} == 0$), the guard tells the customer to wait. This loops back to checking again until a toilet is free. This simulates a process being blocked or waiting on a semaphore. If yes ($\text{Empty} > 0$), the guard allows the customer to proceed.

Guard signal customer to enter a toilet

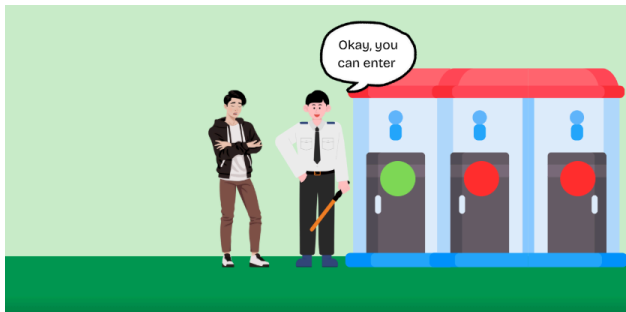


After confirming there is an empty toilet, the guard signals the customer that they may enter once there is an available empty toilet. This is like P (wait) operation in semaphore logic: permission is granted. One toilet is now occupied, so we decrease the Empty count by 1. This ensures we keep track of how many toilets are left.

Customer finishes and exits the toilet



Once done, the customer leaves the toilet, making it available again. This is the end of the critical section. We increase Empty by 1 to reflect that one toilet is now free. This is like the V (signal) operation in semaphore logic.



The process for one customer ends. The guard continues serving the next customer request.

6.0 RESULTS AND DISCUSSION

This section will detail the animation's interface and execution flow, followed with comprehensive analysis of how the animated output effectively illustrates core operating system concepts. The discussion further evaluates the success of using a relatable real-world scenario and animation for understanding complex synchronization mechanisms.

6.1 Interface Execution

The animation serves as the primary interface and execution output for this project where It demonstrates users accessing stalls sequentially, waiting if none are available, and re-entering when stalls are freed.

Visual Interface

The interface consists of a static background depicting a public toilet facility with three individual stalls. A security guard character is positioned in front, and various "person" characters approach and interact with the guard and the stalls. The most crucial visual cue is the changing color of the stall doors: green for available and red for occupied. Dialogue bubbles are used to convey communication between the arriving people and the security guard, representing the conceptual **acquire()** and **release()** requests and responses.

Execution Walkthrough

The animation starts with all three toilet stalls vacant, indicated by green doors. A person approaches the guard, inquires about availability, receives permission, and proceeds to enter a stall, which then turns red. This process repeats for the next two stalls until all three are occupied. When a fourth person arrives and inquires, the guard explicitly states "No" and instructs them to "wait," visually demonstrating the blocking mechanism of the semaphore when resources are exhausted. Subsequently, when a stall becomes free (the door turns green), the guard grants permission to a waiting person, who then enters the newly available stall. The sequence clearly demonstrates the controlled flow of users accessing the limited resource.

6.2 Output Analysis

The animated output effectively demonstrates the principles of semaphore synchronization:

- **Mutual Exclusion:** The animation consistently shows that only one person occupies each stall at any given time. This directly illustrates how the semaphore which is represented by the guard that enforces mutual exclusion, a core requirement for critical sections.
- **Orderly Waiting:** When all stalls are occupied, the animation distinctly shows newly arriving individuals being told to "wait" by the guard and is sustained until a stall becomes available only then the access is granted. This perfectly mimics the blocking and unblocking mechanism of a semaphore's **wait()** operation.
- **Resource Management:** The animation provides a tangible representation of resource management. The fixed number of stalls (resources) are allocated and deallocated in a controlled manner, preventing over-utilization and ensuring fairness among users.
- **Prevention of Race Conditions:** By visually controlling access through the security guard (semaphore), the animation implicitly demonstrates the prevention of race conditions.

6.3 Discussion

The usage of animation serves as an excellent and efficiently simulates semaphore-based synchronization using a relatable and visual approach. It also effectively demonstrates how semaphores manage concurrent access to shared resources, by reinforcing the operating systems theory through an engaging and simplified storytelling. The security guard effectively personifies the semaphore, managing permits and controlling access, making the **acquire()** and **release()** operations easy to visualize.

7.0 FUTURE WORK AND CONCLUSION

7.1 Future Work

The current animation project successfully introduces the semaphore synchronization using the Toilet Stall animation analogy, this is some of the improvements and enhancements that are possible:

- **Interactive Simulation:** Develop a web-based simulation where users can request to use a toilet stall and observe the guard's behaviour, including entering/exiting the stalls. This could provide more hands-on experience.
- **Real Code Integration:** Use a visual overlay of actual C or Python semaphore code like `acquire()` and `release()` that updates live the animation events. This also can directly link the visual representation to the programming concepts.
- **Accessibility Features:** Include the subtitles for better explanation based on the presentation so that viewers can easily understand.

7.2 Conclusion

To conclude, this project successfully demonstrates the fundamental concept of binary semaphore synchronization in operating systems through a relatable real-world scenario. By using the Toilet Stall analogy as a critical section and customers as processes, the animation clearly shows the behaviour of `wait()` and `release()` operations and effectively applying mutual exclusion. The guard acts as the semaphore to ensure only one person can enter the available stall at a time and makes the users wait if no stalls are available. This project enhances the understanding of abstract synchronization, which are important to modern operating systems design.

8.0 ACKNOWLEDGEMENT

First and foremost, we would like to express our deepest gratitude to Dr. Danlami and Dr. Zafran, our lecturers for SECR 2043 – Operating System, for their invaluable guidance, support, and dedication throughout the duration of this project. Their clear explanations and encouragement have greatly enhanced our understanding of synchronization concepts, especially in the context of semaphores which played a key role in the successful completion of our project entitled "Real World Application of Synchronization Using Semaphore." This project has allowed us to explore the practical implementation of semaphores in managing shared resources and has significantly strengthened our grasp on theoretical knowledge through simulation. Lastly, we extend our thanks to everyone who contributed indirectly to the completion of this project. Your support and input are sincerely appreciated.

9.0 REFERENCES

Arm Limited. (n.d.). *What is a semaphore?* Arm Developer.

<https://developer.arm.com/documentation/ka003612/latest>

Downey, A. B. (2016). *The little book of semaphores* (2nd ed., Version 2.2.1) [PDF]. Green Tea Press. <http://greenteapress.com/semaphores>

Jain, S. (2025, January 7). *Semaphores in Process Synchronization*. GeeksforGeeks. Retrieved July 3, 2025, from <https://www.geeksforgeeks.org/operating-systems/semaphores-in-process-synchronization/>

Jain, S. (2025, July 1). *Introduction of Process Synchronization*. GeeksforGeeks. Retrieved July 3, 2025, from <https://www.geeksforgeeks.org/operating-systems/introduction-of-process-synchronization/>

Oracle. (n.d.). Semaphores. In *Multithreaded Programming Guide* (Chapter 4: Programming with Synchronization Objects). Oracle. <https://docs.oracle.com/cd/E19683-01/806-6867/6jfpgcdcnj/index.html>

10.0 YOUTUBE VIDEO LINK

Project Video Link: https://youtu.be/E5oFSc36C_8